

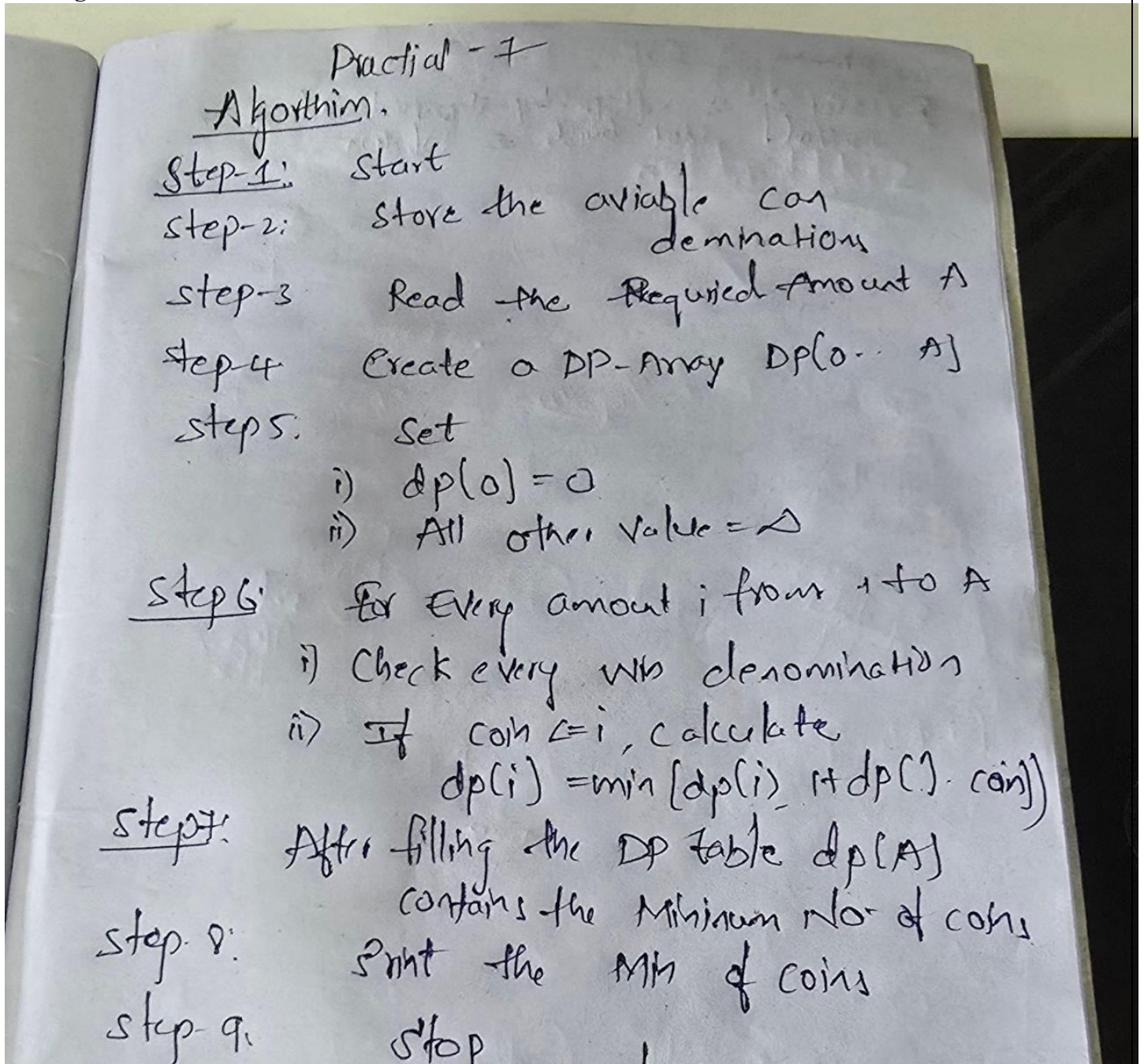
Practical: 7

Implementation of making a change problem using dynamic programming

15/09/2026

Aim: To implement of making a change problem using dynamic programming

Algorithm:



Practical - 7

Algorithm.

Step-1: Start

Step-2: Store the available coin denomination

Step-3: Read the Required Amount A

Step-4: Create a DP-Array $DP[0 \dots A]$

Step-5: Set

- i) $dp[0] = 0$
- ii) All other value = ∞

Step-6: For Every amount i from 1 to A

- i) Check every coin denomination
- ii) If $coin \leq i$, calculate
 $dp[i] = \min(dp[i], 1 + dp[i - coin])$

Step-7: After filling the DP table $dp[A]$ contains the Minimum No. of coins

Step-8: Print the Min of coins

Step-9: Stop

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    // Static coin denominations
    int coins[] = {1, 2, 5, 10};

    int n = 4;
    int amount = 12;

    int dp[100];

    // Base case
    dp[0] = 0;

    // Initialize DP array
    for (int i = 1; i <= amount; i++)
        dp[i] = 9999;

    // Dynamic Programming
    for (int i = 1; i <= amount; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (coins[j] <= i)
            {
                dp[i] = min(dp[i],
                    1 + dp[i - coins[j]]);
            }
        }
    }

    cout << "Coins: 1 2 5 10" << endl;
    cout << "Amount: " << amount << endl;

    cout << "Minimum number of coins = "
        << dp[amount] << endl;

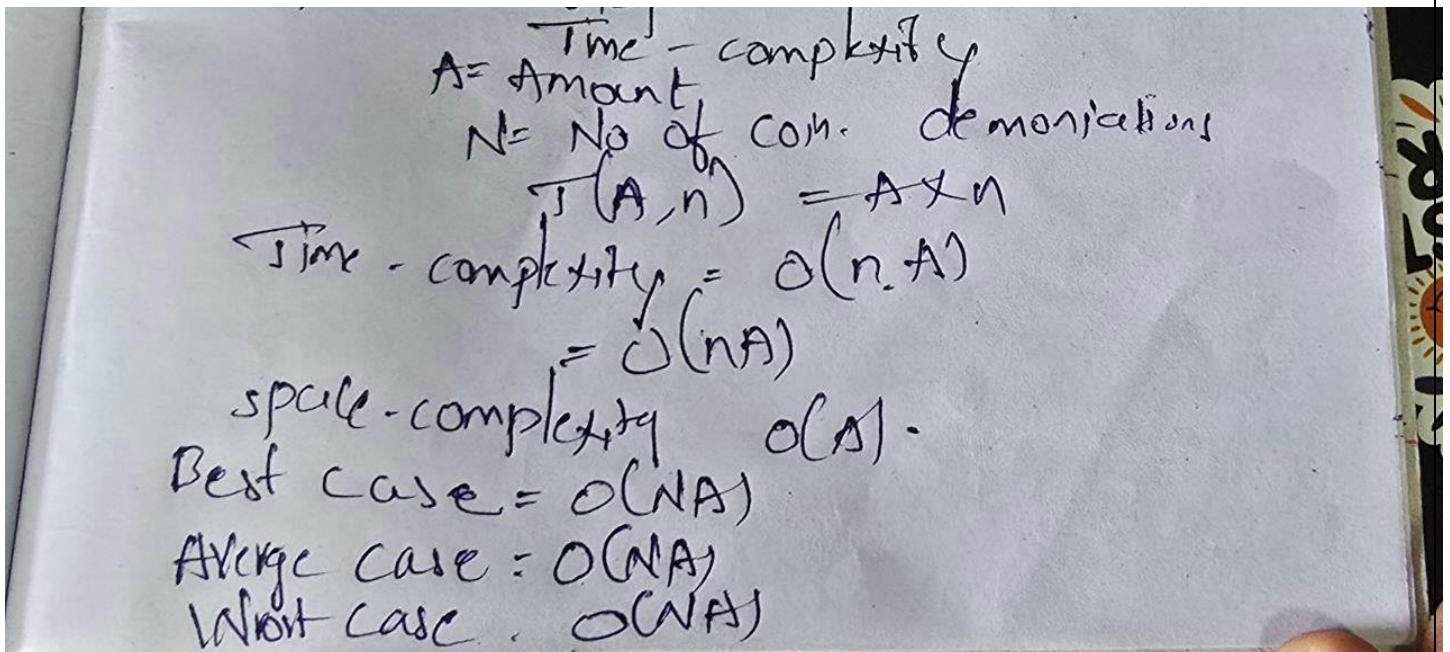
    return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D  X + v
Coins: 1 2 5 10
Amount: 12
Minimum number of coins = 2

-----
Process exited after 0.01435 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:



Conclusion:

The Implementation of making a change problem using dynamic programming is successfully executed with the time complexity of $O(nA)$ along with same for all the three cases.

Best case = Average case = Worst case = $O(nA)$.